10. Classes and Constructors

Exam Objectives

1. Create a new class including a main method (covered in the "Basic Java Elements" chapter)
2. Use the private modifier
3. Describe the relationship between an object and its members
4. Describe the difference between a class variable, an instance variable, and a local variable
5. Develop code that creates an object's default constructor and modifies the object's fields
6. Use constructors with and without parameters
7. Develop code that overloads constructors

10.1 Relationship between an object and its members

10.1.1 Accessing object fields

By object fields, we mean instance variables of a class. Each instance of a class gets its own personal copy of these variables. Thus, each instance can potentially have different values for these variables. To access these variables, i.e., to read the values of these variables or to set these variables to a particular value, we must know the exact instance whose variables we want to manipulate. We must have a reference pointing to that exact object to be able to manipulate that object's fields.

Recall our previous discussion about how an object resides in memory and a reference variable is just a way to address that object. To access the contents of an object or to perform operations on that object, you need to first identify that object to the JVM. A reference variable does just that. It tells the JVM which object you want to deal with.

For example, consider the following code:

```
class Student{
     String name;
}
public class TestClass{
    public static void main(String[] args){
        Student s1 = new Student();
        Student s2 = new Student();

        s1.name = "alice";
        System.out.println(s1.name); //prints alice
        System.out.println(s2.name); //prints null

        s2.name = "bob";
        System.out.println(s1.name); //prints alice
        System.out.println(s2.name); //prints bob
    }
}
```

In the above code, we created two Student objects. We then set the name variable of one Student object and print names of both the Student objects. As expected, the name of the second Student object is printed as null. This is because we never set the second Student object's name variable to anything. The JVM gave it a default value of `null`.

Next, we set the name variable of the second Student object and print both the values again. This time we are able to see the two values stored separately in two Student instances.

This simple exercise shows how to manipulate fields of an object. We take a reference variable and apply the dot operator and the name of the variable to reach that field of the object pointed to by that variable. It is not possible to access the fields of an object if you do not have a reference to that object. You may store the reference to an object in a variable (such as `s1` and `s2` in the code above) when that object is created and then pass that reference around as needed. Sometimes you may not want to keep a reference in a variable. This typically happens when you want to create an object to call a method on it just once. The following piece of code illustrates this:

```
class Calculator{
   public int calculate(int[] iArray){
       int sum = 0;
       for(int i : iArray){ //this is a for-each loop, we'll cover it later
          sum = sum+i;
       }
       return sum;
   }
}
public class TestClass {
  public static void main(String[] args) {
    int result  = new Calculator().calculate( new int[]{1, 2, 3, 4, 5} );
    System.out.println(result);
  }
}
```

Observe that we created two objects in the main method but did not store their references anywhere - a `Calculator` object and an array object. Then we called an instance method on the `Calculator` object directly without having a reference variable. Since the method call is chained directly to the object creation, the compiler is able to create a temporary reference variable pointing to the newly created object and invoke the method using that variable. However, this variable is not visible to the programmer and therefore, after this line, we have lost the reference to the `Calculator` object and there is no way we can access the same `Calculator` object again.

Within the `calculate` method, the same `Calculator` object is available though, through a special variable called "`this`", which is the topic of the next section.

Similarly, the compiler created a temporary reference variable for the array object and passed it in the method call. However, we don't have any reference to this array object after this line and so, we cannot access it anymore. Within the calculate method, however, a reference to that array object is available through the method parameter `iArray`.

10.1.2 What is "this"?

Let us modify our `Student` class a bit:

```
class Student{
   String name;

   public static void main(String[] args) {
      Student s1 = new Student(); //1
      s1.name = "mitchell"; //2
      s1.printName(); //3 prints mitchell
   }

   public void printName(){
     System.out.println(name); //5
   }
```

```
}
```

I mentioned earlier that it is not possible to access the fields of an object without having a reference to that object. But at //5, we are not using any such reference. How is that possible? How is the JVM supposed to know which `Student` instance we mean here?

Observe that at //3, we are calling the `printName` method using the reference variable `s1`. Therefore, when the JVM invokes this method, it already knows the instance on which it is invoking the method. It is the same instance that is being pointed to by s1. Now, in Java, if you don't specify any reference variable explicitly within any instance method, the JVM assumes that you mean to access the same object for which the method has been invoked. Thus, within the `printName` method, the JVM determines that it needs to access the name field of the same `Student` instance for which `printName` method has been invoked. You can also explicitly use this reference to the same object by using the keyword `"this"`. For example, //5 can be written as: `System.out.println(this.name);`

Thus, the rule about having a reference to access the instance fields of an object still applies. Java supplies the reference on its own if you don't specify it explicitly.

> By automatically assuming the existence of the reference variable "this" while accessing a member of an object, Java saves you a few keystrokes. However, it is not considered a good practice to omit it. You should always type "this." even when you know that you are accessing the field of the same object because it improves code readability. The usage of "this." makes the intention of the code very clear and easy to understand.

When is "this" necessary?

When you have more than one variable with the same name accessible in code, you may have to remove the ambiguity by using the `this` reference explicitly. This typically happens in constructors and setter methods of a class. For example,

```
class Student{
   String id;
   String name;
   public void setName(String code, String name){
      id = code;
      name = name;
   }
}
```

In the `setName` method, four variables are accessible: two method parameters - `code` and `name`, and two instance variables - `id` and `name`. Now, within the method code, when you do `id = code;` the compiler knows that you are assigning the value of the method parameter `code` to the instance field `id` because these names refer to exactly one variable each. But when you do `name = name;`, the compiler cannot distinguish between the two `name` variables. It thinks that `name` refers to the method parameter and assigns the value of the method parameter to itself, which is

basically redundant and is not what you want. There is nothing wrong with it from the compiler's perspective but from a logical perspective, the above code has a serious bug. In technical terms, this is called **"shadowing"**. A variable defined in a method (i.e. either in parameter list or as a local variable) shadows an instance or a static field of that class. It is not possible to access the shadowed variable directly using a simple name. The compiler needs more information from the programmer to disambiguate the name.

To fix this, you must tell the compiler that the name on left-hand side of = should refer to the instance field of the Student instance. This is done using "this", i.e., `this.name = name;`.

While we are on the topic of shadowing, I may as well talk about shadowing of static variables of a class by local variables. Here is an example:

```
class Student{
   static int count = 0;
   public void doSomething(){
       int count = 10;
       count = count; //technically valid but logically incorrect
       Student.count = count; //works fine in instance as well as static methods
       this.count = count; //works fine in an instance method
   }
}
```

The above code has the same problem of redundant assignment. The local variable named `count` shadows the static variable by the same name. Thus, inside `doSomething()`, the simple name `count` will always refer to the local variable and not to the static variable. To disambiguate `count`, ideally, you should use `Student.count` if you want to refer to the static variable but if you are trying to use it from an instance method, you can also use `this.count`. Yes, using `this` is a horrible way to access a static variable but it is permissible. I will talk more about static fields and methods in the "Creating and Using Methods" chapter.

Exam Tip

Redundant assignment is one of the traps that you will encounter in the exam. Most IDEs flash a warning when you try to assign the value of a variable to the same variable. But in the exam, you won't get an IDE and so, you must watch out for it by reading the code carefully.

Here are a few quick facts about `this`:

1. `this` is a keyword. That means you can't use it for naming anything such as a variable or a method.

2. The type of `this` is the class (or an enum) in which it is used. For example, the type of `this` in the `printName` method of `Student` class is `Student`.

3. `this` is just like any other local variable that is set to point to the instance on which a method is being invoked. You can copy it to another variable. For example, you can do `Student s3 = this;` in an instance method of `Student` class.

4. You can't modify `this`, i.e., you can't set it to null or make it point to some other instance. It is set by the JVM. In that sense, it is final.

5. `this` can only be used within the context of an instance of a class. This means, it is available in instance initializer blocks, constructors, instance methods, and also within a class. It is not available within a static method and a static block because static methods (and static initializer blocks) do not belong to an object.

10.2 Apply access modifiers

10.2.1 Accessibility

One of the objectives of object-oriented development is to encourage the users of a component (which could be a class, interface, or an enum) to rely only on the agreed upon contract between the user and the developer of the component and not on any other information that the component is not willing to share.

For example, if a component provides a method to compute taxes on the items in a shopping cart, then the user of that component should only pass the required arguments and get the result. It should not try to access internal variables or logic of that class because using such information will tie the user of that component too tightly to that component. Imagine a developer writes the following code for the TaxCalculator class:

```
class TaxCalculator{
   double rate = 0.1;
   double getTaxAmount(double price){
     return rate*price;
   }
}
```

Ideally, users of the above class should use the `getTaxAmount` method but let us say they do not and access the `rate` variable instead, like this:

```
//code in some other class
double price = 95.0;
TaxCalculator tc = new TaxCalculator();
double taxAmt = price*tc.rate;
```

Later on, the developer realizes that "tax rate" cannot be hardcoded to 0.1. It needs to be retrieved from the database and so, the developer makes the following changes to the class:

```
class TaxCalculator{
   //double rate = 0.1; //no more hardcoding
   double getTaxAmount(double price){
     return getRateFromDB()*price;
   }

   double getRateFromDB(){
```

```
    //fetch rate from db using jdbc
  }
}
```

Since there is no `rate` variable present in this class anymore, all other code that is accessing this variable will now fail to compile. Had they stuck to using the `getTaxAmount` method, they would not have had any problem at all. To prevent compilation failure, the developer will now be forced to maintain the `rate` variable in their new code even though this variable is not required by the class anymore. What if the tax rate changes in the database but is not updated in the `TaxCalculator` class's `rate` variable? What if one rogue user updates the rate variable at an inopportune time while another user is using it to compute taxes? Well, in both the cases the users will be computing taxes incorrectly. There will be no error message to make the developer aware of the problem either. This is a serious matter.

But the problem is not just with the users relying on internal details of a class. The `TaxCalculator` class doesn't give any clue as to what features it supports. How are the users supposed to know that they should be using only the `getTaxAmount` method and not the `rate` variable? In other words, the `TaxCalculator` class doesn't make its public contract clear and therefore, it would be unfair to blame only the users of this class. This is where Java's **access modifiers** come into picture.

10.2.2 Access modifiers

Java allows a class and members of a class to explicitly specify who can access the class and the members using three accessibility modifiers: **private, protected,** and **public**. Besides these three, the absence of any access modifier is also considered an access modifier. This is known as **"default"** access. These access modifiers make your intention about a member very clear not only to the users of your class but also to the compiler. The compiler then helps you enforce your intention by refusing to compile code that violates your intention. Here is how these modifiers affect accessibility:

private - A private member is only accessible from within that class. It cannot be accessed by code in any other class.
For example, the problem that I showed you with the `TaxCalculator` class could have been easily avoided if the developer had simply declared the `rate` variable as private. That would make the variable inaccessible from any other class. The compiler would prevent the users of this class from using the rate variable by refusing to compile the code that tried to use it. Since there would have been no dependency on rate variable, the developer could have easily removed this variable without any impact on anyone.

"default" - A member that has no access modifier applied to it is accessible to all classes that belong to the same package. This is irrespective of whether the class trying to access a default member of another class is a sub class or a super class of the other class. If two classes belong to the same package, then they can access each other's default members. This is also called **"package private"** or simply **"package access"**.

protected - A protected member is accessible from two places - if the accessing class belongs to the same package or if the accessing class is a subclass irrespective of the package to which the subclass belongs. The first case is simple because it is exactly the same as default access. All classes

belonging to the same package can access each other's protected members. The second case is not so, simple and requires a deeper understanding of the concepts of inheritance and polymorphism, which are not covered in this exam.

public - A public member is accessible from everywhere. Any code from any class can make use of a public member of another class. For example, the `getTaxAmount` method could have been made public. That would give a clear signal to the users to use this method if they want to compute the tax amount.

If you order the four access modifiers in terms of how restrictive they are, then private is **most restrictive** and public is **least restrictive**. The other two, i.e., **default** and **protected**, lie between these two. Thus, the order of access modifiers from the most restrictive to the least would be: private > default > protected > public.

Exam Tip

In the exam, watch out for non-existent access modifiers such as "friend", "private protected", and "default".

10.3 Apply encapsulation principles to a class

10.3.1 Encapsulation

Encapsulation is considered as one of the three pillars of Object-Oriented Programming. The other two being **Inheritance** and **Polymorphism**. In the programming world, the word **encapsulation** may either refer to a language mechanism for restricting direct access to an object's data fields **or** it may refer to the features of a programming language that facilitate the bundling of data with the methods operating on that data. For the purpose of the exam, we will be focusing on the first meaning, i.e., restricting direct access to an object's fields.

As I explained in the previous section on **access modifiers**, letting other classes directly access instance variables of a class causes **tight coupling** between classes and that reduces the maintainability of the code. While designing a class, your goal should be to present the functionality of this class through methods and not through variables. In other words, a user of your class should be able to make use of your class by invoking methods and not by accessing variables. There are two advantages of this approach:

1. You give yourself the freedom to modify the implementation of the functionality without affecting the users of your class. In fact, users of a well encapsulated class do not even become aware of the internal variables that the class uses for delivering that functionality because the implementation details of the functionality are hidden from the users. In that sense, **encapsulation** and **information hiding** go hand in hand.

2. You can ensure that the value of a variable is always consistent with the business logic of the class. For example, it wouldn't make sense for the `age` variable of a `Person` class to have a negative value. If you make this variable public, anyone could mess with `Person` objects by setting their ages to a negative value. It would therefore, be better if the `Person` class has a public setter method instead, like this:

```
class Person{
   private int age;

   public void setAge(int yrs){
      if(yrs<0) throw new IllegalArgumentException();
     else this.age = yrs;
   }

   public int getAge(){ return age; }

   //other code
}
```

Using access modifiers in professional code

Since the only way to restrict access to the variables of a class from other classes is to make them private, a well encapsulated class defines its variables as private. The more you relax the access restrictions on the variables, the less encapsulated a class gets. Thus, a class with public fields is not encapsulated at all. The visibility of the methods, on other hand, can be anything ranging from private to public, depending on the business purpose of the class.

In your code, you should always act miserly while giving access rights to other classes, i.e., always try to make your class members as restricted (private) as possible.

Accessibility of accessor methods

There is almost never a need to make data fields non-private. If you want other classes to access the data members of your class, you should provide non-private methods that get and set values of those variable. For example in the above code, the age field of Person class is private. The `getAge` and `setAge` are accessor methods and are public.

Make methods non-private only if you are sure that other classes need to access them. Be very careful while make making classes and method public. As explained before, making something public means that you want to allow any class from any application to access and thus, depend on your class and you don't want to create any unnecessary dependencies.

Encapsulation of static members

Encapsulation is an OOP concept and generally applies to the instance members of a class and not to the static members. However, if you understand the spirit of encapsulation, you will notice that all we want to do is to prevent others from inadvertently or incorrectly accessing stuff that they should not be accessing. This is true of static variables also. Thus, it is better to have even the static variables as private. Ideally, the only variables that deserve to be public are the ones that are constants.

10.4 Create and overload constructors

10.4.1 Creating instance initializers

When you ask the JVM to create a new instance of a class, the JVM does four things:

1. First, it checks whether that class has been initialized or not. If not, the JVM loads and initializes the class first.

2. Second, it allocates the memory required to hold the instance variables of the class in the heap space.

3. Third, it initializes these instance variables to their default values (i.e. numeric and char variables to **zero**, boolean variables to **false**, and reference variables to **null**).

4. And finally, the JVM gives that instance an opportunity to set the values of the instance variables as per the business logic of that class by executing code written in special sections of that class. These special sections are: **instance initializers** and **constructors**.

It is only after these four steps are complete that the instance is considered "ready to use". Remember the use of the `new` operator to create instances of a class? The JVM performs the all of the four activities mentioned above and only then returns the reference of the newly created and initialized object to your code.

Out of the four activities listed above, you have already seen the details of the first one in the "Declare and initialize variables" section of "Working with Java Data Types" chapter. The second two activities are performed transparently by the JVM. They don't require the programmer to do anything. The fourth activity, which is the subject of this section, depends on the programmer because it involves the code written by the programmer.

Although **instance initializers** are not on the exam, let us take a brief look at them anyway because these are the ones that are executed by the JVM first.

Creating instance initializers

Instance initializers are blocks of code written directly within the scope of a class. Here is an example:

```
class TestClass{

  {
    System.out.println("In instance initializer");
  }

}
```

Observe that there is no method declaration or anything but just a line of code nested inside the opening and closing curly braces. Code inside an instance initializer block is regular code. There is

no limitation on the number of statements or the kind of statements that an instance initialize can have. You may have any number of such instance initializer blocks in a class. The JVM executes them in the order that they appear in the class. The following code, for example, prints `Hello World!` using two instance initializers in different places in a class:

```
class TestClass{

 //first instance initializer
  {
    System.out.print("Hello ");
  }

  public static void main(String[] args){
    new TestClass();
  }

 //second instance initializer
  {
    System.out.print("World!");
  }

}
```

Observe that the main method does nothing except create an instance of TestClass. As a result of this creation, the JVM executes each of the two instance initializers and then returns the newly created instance to the main method. Of course, the main method does not assign the reference of the newly created instance to any variable but that is okay.

Accessing members from instance initializers

Instance initializers have access to all of the members of a class. This includes static as well as instance fields and methods. Just like the instance methods, instance initializers have access to the implicit variable "**this**".

10.4.2 Creating constructors

Constructor of a class

A constructor of a class looks very much like a method of a class but there are two things that make a method a constructor:

1. **Name** - The name of a constructor is always exactly the same as the name of the class.

2. **Return type** - A constructor does not have a return type. It cannot even say that it returns void.

The following is an example of a class with a constructor.

```
class TestClass{

   int someValue;
   String someStr;

   TestClass(int x)  //No return type specified
   {
      this.someValue = x; //initializing someValue
      //return; //legal but not required
   }
}
```

Observe that the name of the constructor is the same as that of the class and that there is no return type in the declaration of the constructor. Also observe that there is no return statement because a constructor cannot return anything, not even void. However, it is permissible to write an empty return statement as shown in the code above.

It is interesting to note that a class can have a method with the same name as that of the class. For example, consider the following class:

```
class TestClass{

   int someValue;
   String someStr;

   void TestClass(int x) //<-- observe the return type void
   {
      this.someValue = x;
   }
}
```

In the above code, `void TestClass(int x)` is not a constructor but it is a valid method nevertheless and so the code will compile fine.

Besides the above two rules there are several other rules associated with constructors. Some of them are related to inheritance and exception handling and I will cover them later. Here, I will talk about rules that are applicable to constructors in general.

The default constructor

If I tell you that every class must have a constructor, you might have trouble believing it because so far, I have shown you so many classes that had no constructor at all! Well, here is the deal - it is true that every class must have at least one constructor but the thing is that the programmer doesn't necessarily have to provide one. If the programmer doesn't provide any constructor, then the compiler will add a constructor to the class on its own. This constructor, that is, the one provided by the compiler, is called the **"default"** constructor. This default constructor takes no argument and has no code in its body. In other words, it does absolutely nothing. For example, if I write and compile this class, `class Account { }`, the compiler will automatically add a constructor to this class which looks like this:

```
class Account{
    Account(){ } //default constructor added by the compiler
}
```

This sounds simple but the cause of confusion is the fact that if you write any constructor in a class yourself, the compiler will not provide the default constructor at all. So, for example, consider the following class:

```
class Account{
    int id;
    Account(int id){
       this.id = id;
    }
    public static void main(String[] args) {
       Account a =  new Account(); //trying to use the no-args constructor
    }
}
```

I have provided a constructor explicitly in the above class. Can you can guess what will happen if I try to compile this class? It will not compile. The compiler will complain that `Account` class does not have a constructor that takes no arguments. What happened to the default constructor, you ask? Well, since this class provides a constructor explicitly, the compiler feels no need to add one on its own.

The discussion on default constructors also gives me an opportunity to talk about one misconception that I have often heard, which is that constructors must initialize all instance members of the class. This is not true. A constructor is provided by you, the programmer, and you can decide which instance members you want to initialize. For example, the constructor I showed earlier for `TestClass` doesn't touch the `someStr` variable. The default constructor provided by the compiler also doesn't assign any values to the instance members. Remember that the JVM always provides default values to static and instance members anyway.

Exam Tip

You will most certainly get a question that tests your knowledge about the default constructor. Watch out for code that assumes the existence of the default constructor while the class provides a constructor explicitly.

Difference between default and user defined constructors

There are only two things that you need to remember about a default constructor:

1. **When is it provided** - It is provided by the compiler **only** when a class does not define **any** constructor explicitly.

2. **What does it look like** - The default constructor is the simplest constructor that you will ever see. It does not take any argument, does not have any throws clause, and does not

contain any code.
But one peculiar thing about the default constructor is its accessibility. The accessibility of the default constructor is the same as that of the class. That means, if the class is public, the default constructor will also be public and if the class has default accessibility, the default constructor will also have default accessibility. If you have no idea about accessibility, don't worry, I will talk about it in the next topic.

The default constructor is also sometimes called the **"default no-args"** constructor because it does not take any arguments. But this is technically imprecise because the default constructor is always a no-args constructor. There is nothing like a default constructor with args!

You can, on the other hand, write a no-args constructor explicitly in a class with a throws clause and with a different accessibility. For example, it is quite common to have a class with private no-args constructor if you don't want anyone to create instances of that class. I will talk more about it in the next topic.

Benefit of constructors

Anything that you can do in a method, you can do in a constructor and vice-versa. There is no limitation on what a method or a constructor can do. Why not just have a method and name it as init() or something? Indeed, you will encounter Java frameworks such as servlets that do exactly that. Then why do you need a constructor? Well, a detailed discussion of the pros and cons of constructors is beyond the scope of this book, but I will give you a couple of pointers.

As I mentioned earlier, an instance is not considered "ready to use" until its constructor finishes execution. Thus, a constructor helps you make sure that an instance of your class will be initialized as per your needs before anyone can use it. Here, "use" essentially means other code accessing that instance through its fields or methods.

It is possible to do the same initialization in a method instead of a constructor but in that case the users of your class would have to remember to invoke that method explicitly after creating an instance. What if a user of your class fails to invoke that special method after creating an instance of your class? The instance may be in a logically inconsistent state and may produce incorrect results when the user invokes other methods later on that instance. Therefore, a constructor is the right place to perform all initialization activities of an object. It is a place where you make sure that the instance is ready with all that it needs to perform the activities it is supposed to perform in other methods.

Constructors also provide thread safety because no thread can access the object until the constructor is finished. This protection is guaranteed by the JVM to the constructors and is not always available to methods.

10.4.3 Overloading constructors

A class can have any number of constructors as long as they have different signatures. Since the name of a constructor is always the same as that of the class, the only way you can have multiple constructors is if their parameter type list is different.

Just like methods with same names, if a class has more than one constructor, then this is called **"constructor overloading"** because the constructors are different only in their list of parameter types.

There are a couple of differences between method overloading and constructor overloading though. Recall that in case of method overloading, a method can call another method with the same name just like it calls any other method, i.e., by using the method name and passing the arguments in parenthesis. In case of constructor overloading, when a constructor calls another constructor of the same class, it is called **"constructor chaining"** and it works a bit differently. Let me show you how.

Constructor chaining

A constructor can invoke another constructor using the keyword **this** and the arguments in parenthesis. Here is an example:

```
class Account{
    int id;
    String name;
    Account(String name){
       this(111, name); //invoking another constructor here
       System.out.println("returned from two args constructor");
    }
    Account(int id, String name){
       this.id = id;
       this.name = name;
    }
    public static void main(String[] args) {
       Account a =  new Account("amy");
    }
}
```

Observe that instead of using the name of the constructor, the code uses 'this'. That is, instead of calling `Account(111, name);`, the code calls `this(111, name);` to invoke the other constructor. It is in fact against the rules to try to invoke another constructor using the constructor name and it will result in a compilation error.

Invoking another constructor from a constructor is a common technique that is used to initialize an instance with different number of arguments. As shown in the above example, the constructor with one argument calls the constructor with two arguments. It passes one user supplied value and one default value to the second constructor. This helps in keeping all initialization logic in one constructor, while allowing the user of the class to create instances with different parameters.

The only restriction on the call to the other constructor is that it must be the first line of code in a constructor. This implies that a constructor can invoke another constructor only once at the most. Thus, the following three code snippets for constructors of Account class will not

compile:

```
Account(String name){
    System.out.println("calling two args constructor");
    this(111, name); //call to another constructor must be the first line
}

Account(){
    this(111);
    this(111, "amy"); //call to another constructor must be the first line
}

Account(String name){
   Account(111, name); //incorrect way to call to another constructor but if the
    Account class had a method named Account, this would be a valid call to that method
}
```

Invoking a constructor

It is not possible to invoke the constructor of a class directly. It is invoked only as a result of creation of a new instance using the "new" keyword. The only exception to this rule is when a constructor invokes another constructor through the use of "this" (and "super", which I will talk about in another chapter) as explained above. In other words, if you are given a reference to an object, you cannot use that reference to invoke the constructor on that object.

Now, consider the following program. Can you tell which line out of the four lines marked LINE A, LINE B, LINE C, and LINE D should be uncommented so that it will print `111, dummy`?

```
class Account{
   int id;
   String name;
   public Account(){
      id = 111;
      name = "dummy";
   }

   public void reset(){
     //this(); //<-- LINE A
     //Account(); //<-- LINE B
     //this = new Account(); //<-- LINE C
     //new Account(); //<-- LINE D
   }

   public static void main(String[] args) {
       Account a =  new Account();
       a.id = 2;
       a.name = "amy";
       a.reset();
```

```
        System.out.println(a.id+", "+a.name);
    }

}
```

The answer is none of these. First three are invalid attempts to invoke the constructor - `this()` can only be used within a constructor, `Account()` is interpreted as method call to a method named Account, but such a method doesn't exist in the given code, and `this = new Account()` attempts to change "`this`", which is a final variable, to point to another object. Thus, none of these three statements will compile.
`new Account()` is a valid statement but it creates an entirely new Account object. The instance variables of this new Account object are indeed set to `111` and "`dummy`", but this doesn't change the values of the current Account object.

10.5 Understanding the scopes of class, instance, and local variables

10.5.1 Scope of variables

Java has three **visibility scopes** for variables - class, method, and block.

Java has five **lifespan scopes** for variables - class, instance, method, for loop, and block.

10.5.2 Scope and Visibility

Scope means where all, within a program, a variable is visible or accessible directly without any using any referencing mechanism.

For example, the scope of a President of a country is that country. If you say "The President", it will be interpreted as the person who is the president of the country you are in. There cannot be two presidents **of** a country. If you really want to refer to the presidents of two countries, you must also specify the name of the country. For example, the President of US and the President of India.

At the same time, you can certainly have two presidents **in** a country - the President of the country, and the President of a basketball association within that country! If you are in your basketball association meeting, and if you talk about the president in that meeting, it will be interpreted as person who is the president of the association and not the person who is the president of your country. But if you do want to mean the president of the country, you will have to clearly say something like the "President of our country". Here, "of our country" is the referencing mechanism that removes the ambiguity from the word "president".

In this manner, you may have several "presidents" in a country. All have their own "visibility". Depending on the context, one president may shadow or hide (yes, the two words have different meanings in Java) another president. But you cannot have two presidents in the same "visibility" level.

This is exactly how "scope" in Java (or any other programming language, for that matter) works. For example, if you declare a static variable in a class, the **visibility** of that variable is the class in which you have defined it. The visibility of an instance variable is also the class in which it is

defined. Since both have same visibility, you cannot have a static variable as well as an instance variable with the same name in the same class. It would be like having two presidents of a country. It would be absurd and therefore, invalid.

If you declare a variable in a method (either as a method parameter or within a method), the visibility of that variable is within that method only. Since a method scope is different from a class scope, you can have a variable with the same name in a method. If you are in the method and if you try to refer to that variable directly, it will be interpreted as the variable defined in the method and not the class variable. Here, a method scoped variable **shadows** a class scoped variable. You can, of course, refer to a class scoped variable within a method in such a case but you would have to use the class name for a static variable or an object reference for an instance variable as a referencing mechanism to do that. **Hiding** is similar, where a subclass variable hides the variable by the same name in a superclass. Since it involves inheritance, I will discuss it in detail later.

Similarly, if you declare a variable in a loop, the visibility of that variable is only within that loop. If you declare a variable in a block such as an if, do/while, or switch, the visibility of that variable is only within that block.

Here, visibility is not to be confused with **accessibility** (public/private/protected). Visibility refers to whether the compiler is able to see the variable at a given location directly without any help.

For example, consider the following code:

```
public class Area{
    public static String UNIT="sq mt"; //UNIT is visible all over inside the class Area
    public void printUnit(){
        System.out.println(UNIT); //will print "sq mt" because UNIT is visible here
    }
}

public class Volume{
    //Area's UNIT is accessible in this class but not visible to the compiler directly
    public static String UNIT="cu mt";

    public void printUnit(){
        System.out.println(UNIT); //will print "cu mt"
        System.out.println(Area.UNIT); //will print "sq mt"
    }
}
```

In the above code, a public static variable named `UNIT` of a class `Area` is accessible to all other classes but that doesn't mean another class `Volume` cannot also have a static variable named `UNIT`. This is because within the `Volume` class, `Area`'s `UNIT` is not directly visible. You would need to help the compiler by specifying `Area.UNIT` if you want to refer to `Area`'s `UNIT` in class Volume. Without this help, the compiler will assume that you are talking about `Volume`'s `UNIT`.

Besides shadowing and hiding, there is a third category of name conflicts called "obscuring". It happens when the compiler is not able to determine what a simple name refers to. For example, if a class has a field whose name is the same as the name of a package and if you try to use that simple name in a method, the compiler will not know whether you are trying to refer to the field or to a member of the package by the same name and will generate an error. It happens rarely and is not important for the exam.

10.5.3 Scope and Lifespan

Scope and Lifespan

Besides visibility, **scope** is also related to the **lifespan** or **life time** of a variable. Think of it this way - what happens to the post of the president of your local basketball association if the association itself is dissolved? The post of the president of the association will not exist anymore, right? In other words, the life of the post depends on the existence of the association.

Similarly, in Java, the existence of a variable depends on the existence of the scope to which it belongs. Once its life time ends, the variable is destroyed, i.e., the memory allocated for that variable is taken back by the JVM. From this perspective, Java has five scopes: **block, for loop, method, instance,**and **class.**

When a block ends, variables defined inside that block cease to exist. For example,

```
public class TestClass
{
  public static void main(String[] args){

    {
       int i = 0; //i exists in this block only
       System.out.println(i); //OK
     }
     System.out.println(i); //NOT OK because i has already gone out of scope
  }

}
```

Variables defined in a for loop's initialization part exist as long as the for loop executes. Notice that this is different from variables defined inside a for block, which cease to exist after each iteration of the loop. For example,

```
public class TestClass
{
  public static void main(String[] args){
    for(int i = 0; i<10; i++)
    {
       int k = 0; //k is block scoped. It is reset to 0 in each iteration
       System.out.println(i); // i retains its value from previous iteration
     }
```

```
    //i and k are both out of scope here
  }

}
```

When a method ends, the variables defined in that method cease to exist.

When an object ceases to exist, the instance variables of that object cease to exist.

When a class is unloaded by the JVM (not important for the exam), the static variables of that class cease to exist.

It is important to note here that lifespan scope doesn't affect compilation. The compiler checks for the visibility scope only. In case of blocks, loops, and, methods, the lifespan scope of the variables coincides with the visibility scope. But it is not so for class and instance variables. Here is an example:

```
public class TestClass
{
  int data = 10;
  public static void main(String[] args){
     TestClass t = new TestClass();
     t = null;
     System.out.println(t.data); //t.data is accessible therefore, it will compile fine
    even though the object referred to by t has already ceased to exist
  }

}
```

Lifespan scope affects the run time execution of the program. For example, the above program throws a `NullPointerException` at run time because `t` doesn't exist and neither does `t.data` at the time we are trying to access `t` and `t.data`.

10.5.4 Scopes Illustrated

The following code shows various scopes in action.

```
class Scopes{
    int x; //visible throughout the class
    static int y; //visible throughout the class

    public static void method1(int param1){ //visible throughout the method
        int local1 = 0; //visible throughout the method

        {
            int anonymousBlock = 0; //visible in this block only
        }

        anonymousBlock = 1; //compilation error

        for(int loop1=0; loop1<10; loop1++){
```

```
            int loop2 = 0;
            //loop1 and loop2 are visible only here
        }
        loop1 = 0; //compilation error
        loop2 = 0; //compilation error
        if(local1==0){
            int block1 = 0; //visibile only in this if block
        }

        block1 = 7; //compilation error

        switch(param1){
            case 0:
                int block2 = 10; //visible all over case block
                break;
            case 1:
                block2  = 5; //valid
                break;
            default:
                System.out.println(block2); //block2 is visible here but compilation
    error because block2 may be left uninitialized before access
        }
        block2 = 9; //compilation error
        int loop1 = 0, loop2 = 0, block1 = 0, block2 = 8; //all valid
    }
}
```

10.5.5 Scope for the Exam

The important thing about scopes that you must know for the exam is when you can and cannot let the variable with different scopes overlap. A simple rule is that you cannot define two variables with the same name and same visibility scope. For example, check out the following code:

```
class Person{
  private String name; //class scope

  static String name = "rob"; //class scope. NOT OK because name with class scope
    already exists

  public static void main(String[] args){
     for(int i = 0; i<10; i++){
          String name = "john"; //OK. name is scoped only within this for loop block
     }
     String name = "bob"; //OK. name is method scoped
     System.out.println(name); //will print bob
  }
```

```
}
```

In the above code, the static and instance name variables have the same visibility scope and therefore, they cannot coexist. But the name variables inside the method and inside the for loop have different visibility scopes and can therefore, coexist.

But there is an **exception** to this rule. Consider the following code:

```
class Person{
  private String name; //name is class scoped

  public static void main(String[] args){
     String name = "bob"; //method scope. OK. Overlaps with the instance field name
    defined in the class
     int i = -1; //method scope

     for(int i = 0; i<10; i++){ //i has for loop scope. Not OK
          String name = "john"; //block scope. Not OK.
     }

     {  //starting a new block here
         int i = 2; //block scope. Not OK.
     }
  }
}
```

Observe that it is possible to overlap the instance field with a method local variable but it is not possible to overlap a method scoped variable with a loop or block scoped variable.

10.5.6 Quiz

Q1. What will the following code print when compiled and run?

```
public class TestClass {
    public static void main(String[] args) {

         {
            int x = 10;
         }
         System.out.println(x);
    }

}
```

Select 1 correct option

A. It will not compile.

B. It will print 10

C. It will print an unknown number

The correct answer is A.
Notice that x is defined inside a block. It is not visible outside that block. Therefore, the line `System.out.println(x);` will not compile.

Q.2 What will the following code print when compiled and run with the command line:

```
java ScopeTest hello world
```

```
public class ScopeTest {
    private String[] args = new String[0];

    public static void main(String[] args) {
        args = new String[args.length];
        for(String arg: args){
            System.out.println(arg);
        }
        String arg = args[0];
        System.out.println(arg);
    }

}
```

Select 1 correct option

A. It will not compile.

B. It will print:

```
null
null
null
```

C. It will print:

```
null
null
hello
```

Answer is B.
The line `args = new String[args.length];` creates a new string array with the same length as the length of the original string array passed to the program and assigns it back to the same variable `args`. All the elements of this new array are `null`.
The original string array passed to the program is lost.

The instance variable `args` is not touched here because it is shadowed in the method code by the method parameter named `args`. Also, you need to have a reference to an object of class ScopeTest to access an instance variable from a static method.

10.6 Exercise

1. Define a reference type named `Bird`. Define an instance method named `fly` in `Bird`. Define a few instance as well as static variables of type `int`, `float`, `double`, `boolean`, and `String` in `Bird`.

2. Create a `TestClass` that has a static variable of type `Bird`. Initialize this variable with a valid `Bird` object. Print out the default values of static and instance variables of `Bird` from the `main` method of `TestClass`. Also print out the static variable of `TestClass` from `main`. Observe the output.

3. Create and initialize one more instance variable of type `Bird` in `TestClass`. Assign values to the members of the Bird instance pointed to by this instance variable in `TestClass`'s `main`. Assign values to the members of first `Bird` using the second `Bird`. Print the values of the members of both the `Bird` objects.

4. Write code in `fly` method to print out the values of all members of `Bird`. Alter `main` method of `TestClass` to invoke `fly` on both the instances of `Bird`. Observe the values printed for static variables of `Bird`.

5. Add an instance variable of type `Bird` in `Bird`. Initialize this variable on the same line using "new `Bird()`" syntax. Instantiate a `Bird` object in `TestClass`'s `main` and execute it. Observe the output.

6. Remove the initialization part of the variable that you added to `Bird` in previous exercise. Initialize it with a new `Bird` object in `TestClass`'s `main`. Identify how many `Bird` objects will be garbage collected when the `main` method ends.

7. Add a parameter of type `Float` to `Bird`'s `fly` method. Return an `int` value from `fly` by casting the method parameter to `int`. Invoke `fly` multiple times from `TestClass`'s `main` by passing a `float` literal, a `Float` object, a `double` literal, an `int`, an `Integer`, and a `String` containing a `float` value. Observe which calls compile.

8. Assign the return value of `fly` to an `int` variable, a `float` variable, a `String` variable, and `boolean` variable. Observe which assignments compile. Try the same assignments with an explicit cast. Print these variables out and observe the output.